```
import pandas as pd
import numpy as np

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    StandardScaler, MinMaxScaler,
    OneHotEncoder, LabelEncoder, OrdinalEncoder
)
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.decomposition import PCA
from sklearn.feature_selection import SelectKBest, chi2

from scipy import stats
```

```
data = pd.DataFrame({
    'name': ['John', 'Jane', 'Jack', 'John', None],
    'age': [28, 34, None, 28, 22],
    'purchase_amount': [100.5, None, 85.3, 100.5, 50.0],
    'date_of_purchase': ['2023/12/01', '2023/12/02', '2023/12/01', '2023/12/01', '2023/12/03']
})

data
```

	name	age	purchase_amount	date_of_purchase
0	John	28.0	100.5	2023/12/01
1	Jane	34.0	NaN	2023/12/02
2	Jack	NaN	85.3	2023/12/01
3	John	28.0	100.5	2023/12/01
4	None	22.0	50.0	2023/12/03

Next steps: Generate code with data New interactive sheet

```
imputer = SimpleImputer(strategy='mean')
data[['age', 'purchase_amount']] = imputer.fit_transform(
    data[['age', 'purchase_amount']]
)

data
```

	name	age	purchase_amount	date_of_purchase
0	John	28.0	100.500	2023/12/01
1	Jane	34.0	84.075	2023/12/02
2	Jack	28.0	85.300	2023/12/01
3	John	28.0	100.500	2023/12/01
4	None	22.0	50.000	2023/12/03

Next steps: Generate code with data New interactive sheet

```
data = data.drop_duplicates()
data
```

	name	age	purchase_amount	date_of_purchase
0	John	28.0	100.500	2023/12/01
1	Jane	34.0	84.075	2023/12/02
2	Jack	28.0	85.300	2023/12/01
4	None	22.0	50.000	2023/12/03

Next steps: Generate code with data New interactive sheet

```
data['date_of_purchase'] = pd.to_datetime(data['date_of_purchase'], errors='coerce')
print(data)
```

```
   name   age  purchase_amount date_of_purchase
0  John  28.0          100.500       2023-12-01
1  Jane  34.0           84.075       2023-12-02
2  Jack  28.0           85.300       2023-12-01
4  None  22.0           50.000       2023-12-03
/tmp/ipython-input-271/2231915229.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-ver
  data['date_of_purchase'] = pd.to_datetime(data['date_of_purchase'], errors='coerce')
```

Data Cleaned

Creating Multiple Datasets

```
data1 = pd.DataFrame({
    'customer_id': [1, 2, 3],
    'name': ['John', 'Jane', 'Jack'],
    'age': [28, 34, 29]
})

data2 = pd.DataFrame({
    'customer_id': [1, 3, 4],
    'purchase_amount': [100.5, 85.3, 45.0],
    'purchase_date': ['2023-12-01', '2023-12-02', '2023-12-03']
})

data
```

	name	age	purchase_amount	date_of_purchase
0	John	28.0	100.500	2023-12-01
1	Jane	34.0	84.075	2023-12-02
2	Jack	28.0	85.300	2023-12-01
4	None	22.0	50.000	2023-12-03

Next steps: Generate code with data New interactive sheet

```
merged_data = pd.merge(data1, data2, on='customer_id', how='inner')
merged_data
```

	customer_id	name	age	purchase_amount	purchase_date
0	1	John	28	100.5	2023-12-01
1	3	Jack	29	85.3	2023-12-02

Next steps: Generate code with merged_data New interactive sheet

Data Transformation :- Creating Dataset

```
data = pd.DataFrame({
    'category': ['A', 'B', 'A', 'C', 'B'],
    'numeric_column': [10, 15, 10, 20, 15]
})

data
```

	category	numeric_column
0	A	10
1	B	15
2	A	10
3	C	20
4	B	15

Next steps: Generate code with data New interactive sheet

Scale Numerical Data

```
scaler = StandardScaler()
data['scaled_numeric_column'] = scaler.fit_transform(
    data[['numeric_column']]
)

data
```

	category	numeric_column	scaled_numeric_column
0	A	10	-1.069045
1	B	15	0.267261
2	A	10	-1.069045
3	C	20	1.603567
4	B	15	0.267261

Next steps: Generate code with data New interactive sheet

One-Hot Encode Categorical Data

```
encoder = OneHotEncoder(sparse_output=False)

encoded_data = pd.DataFrame(
    encoder.fit_transform(data[['category']]),
    columns=encoder.get_feature_names_out(['category'])
)

data = pd.concat([data, encoded_data], axis=1)
data
```

	category	numeric_column	scaled_numeric_column	category_A	category_B	category_C
0	A	10	-1.069045	1.0	0.0	0.0
1	B	15	0.267261	0.0	1.0	0.0
2	A	10	-1.069045	1.0	0.0	0.0
3	C	20	1.603567	0.0	0.0	1.0
4	B	15	0.267261	0.0	1.0	0.0

Next steps: Generate code with data New interactive sheet

Data Reduction :- Create Dataset

```
data = pd.DataFrame({
    'feature1': [10, 20, 30, 40, 50],
    'feature2': [1, 2, 3, 4, 5],
    'feature3': [100, 200, 300, 400, 500],
    'target': [0, 1, 0, 1, 0]
})
data
```

	feature1	feature2	feature3	target
0	10	1	100	0
1	20	2	200	1
2	30	3	300	0
3	40	4	400	1
4	50	5	500	0

Next steps: Generate code with data New interactive sheet

Feature Selection (SelectKBest)

```
selector = SelectKBest(score_func=chi2, k=2)
selected_features = selector.fit_transform(
    data[['feature1', 'feature2', 'feature3']],
    data['target']
)

print("Selected Features:")
print(selected_features)
```

```
Selected Features:
[[  1 100]
 [  2 200]
 [  3 300]
 [  4 400]
 [  5 500]]
```

Dimensionality Reduction (PCA)

```
pca = PCA(n_components=2)
pca_data = pca.fit_transform(
    data[['feature1', 'feature2', 'feature3']]
)

print("PCA Output:")
print(pca_data)
```

```
PCA Output:
[[-2.01007463e+02 -2.69618663e-15]
 [-1.00503731e+02  8.98728878e-16]
 [ 0.00000000e+00 -0.00000000e+00]
 [ 1.00503731e+02 -8.98728878e-16]
 [ 2.01007463e+02 -1.79745776e-15]]
```

Handling Missing Data (Techniques)

```
data = pd.DataFrame({
    'column_with_missing': [10, None, 30, None, 50]
})

imputer = SimpleImputer(strategy='mean')
data['column_with_missing'] = imputer.fit_transform(
    data[['column_with_missing']]
)

print(data)
```

```
   column_with_missing
0                 10.0
1                 30.0
2                 30.0
3                 30.0
4                 50.0
```

Deletion

```
data = pd.DataFrame({
    'A': [1, None, 3],
```

```
    'B': [4, 5, None]
})

data.dropna(inplace=True)
print(data)
```

```
     A    B
0  1.0  4.0
```

Outlier Detection :- Z-Score Method

```
data = pd.DataFrame({'column': [10, 12, 14, 100, 13]})

z_scores = np.abs(stats.zscore(data['column']))
outliers = z_scores > 3

print("Outliers detected:")
print(data[outliers])
```

```
Outliers detected:
Empty DataFrame
Columns: [column]
Index: []
```

IQR Method

```
Q1 = data['column'].quantile(0.25)
Q3 = data['column'].quantile(0.75)
IQR = Q3 - Q1

outliers = data[
    (data['column'] < Q1 - 1.5 * IQR) |
    (data['column'] > Q3 + 1.5 * IQR)
]

print(outliers)
```

```
   column
3     100
```

Data Encoding :- One-Hot Encoding

```
data = pd.DataFrame({'color': ['red', 'blue', 'green']})

encoder = OneHotEncoder(sparse_output=False)
encoded = encoder.fit_transform(data[['color']])

print(encoded)
```

```
[[0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]]
```

Label Encoding

```
le = LabelEncoder()
data['color_encoded'] = le.fit_transform(data['color'])

print(data)
```

```
   color  color_encoded
0    red              2
1   blue              0
2  green              1
```

Ordinal Encoding

```
data = pd.DataFrame({'size': ['low', 'medium', 'high']})

oe = OrdinalEncoder(categories=[['low', 'medium', 'high']])
data['size_encoded'] = oe.fit_transform(data[['size']])
```

```
     size  size_encoded
0     low           0.0
1  medium           1.0
2    high           2.0
```

Scaling & Normalization :- Min-Max Scaling

```
data = pd.DataFrame({'value': [10, 20, 30]})

scaler = MinMaxScaler()
data['scaled'] = scaler.fit_transform(data[['value']])

print(data)
```

```
   value  scaled
0     10     0.0
1     20     0.5
2     30     1.0
```

Standardization

```
scaler = StandardScaler()
data['standardized'] = scaler.fit_transform(data[['value']])

print(data)
```

Preprocessing Pipeline (FINAL)

```
import numpy as np
data = pd.DataFrame({
    'age': [25, np.nan, 30],
    'category': ['A', 'B', np.nan]
})

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer

numeric_pipeline = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])

categorical_pipeline = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_pipeline, ['age']),
    ('cat', categorical_pipeline, ['category'])
])

processed_data = preprocessor.fit_transform(data)
print(processed_data)
```

```
[[-1.22474487  1.          0.        ]
 [ 0.          0.          1.        ]
 [ 1.22474487  1.          0.        ]]
```